

JSS MAHAVIDYAPEETHA

JSS SCIENCE AND TECHNOLOGY UNIVERSITY
JSS TECHNICAL INSTITUTIONS CAMPUS MYSURU –
570006 KARNATAKA, INDIA



Department of Information Science and Engineering

An event Report titled

Lossless Compression Algorithms

Submitted by

Madhumica S 01JST24UIS047

Lakshya Srivastava 01JST24UIS089

Under the supervision of

Dr. Vidyashree L

Assistant Professor

Department of I S & E

JSSS&T University, Mysuru.

2025-26

SRI JAYACHAMARAJENDRA COLLEGE OF ENGINEERING



- ☐ Constituent College of JSS Science and Technology University
- ☐ Approved by A.I.C.T.E
- ☐ Governed by the Grant-in-Aid Rules of Government of Karnataka
- ☐ Identified as lead institution for World Bank Assistance under TEQIP Scheme

CERTIFICATE

This is to certify that the work entitled "Lossless Compression Algorithms" is a Bonafide work carried out by Madhumica S and Lakshya Srivastava in partial fulfilment of curriculum prescribed for the award of the Bachelor of Engineering in INFORMATION SCIENCE AND ENGINEERING of Sri Jayachamarajendra College of Engineering, JSS Science and Technology University, Mysuru during the year 2026. It is certified that all suggestions/corrections indicated during CIE have been incorporated in the report. The report has been approved as it satisfies the academic requirements in respect of report work prescribed for the Bachelor of Engineering degree.

NAME	USN	PRESENTATION	REPORT	TOTAL
Madhumica S	01JST24UI047			
Lakshya Srivastava	01JST24UIS089			

ABSTRACT

Digital systems generate, transmit, and store massive volumes of data daily. Managing server storage and optimizing network bandwidth manually or relying on inefficient data handling can cause significant system latency and storage overhead. This project, "Lossless Compression Algorithms" focuses on automating and optimizing file size management using efficient compression techniques studied in Design and Analysis of Algorithms (ADA).

The system accepts files of varying sizes as input and applies lossless compression algorithms specifically Huffman Coding, Gzip (DEFLATE), and Brotli to compress or decompress data without any loss of integrity. The processed files can then be analyzed for performance evaluation and reduction metrics. The project explicitly evaluates the performance of these distinct algorithms by analyzing their execution time, compressed file size, and asymptotic time complexity.

By implementing these algorithmic workflows, the system demonstrates the practical application of data encoding and dictionary-based approaches in modern software engineering. The project highlights the critical importance of selecting suitable algorithms based on dataset characteristics, providing a scalable, interactive full-stack solution for digital asset optimization.

TABLE OF CONTENT

1.ABSTRACT	3
2.PROBLEM STATEMENT	5
3.OBJECTIVES	5
4.PROPOSED SOLUTION	6
5.ARCHITECTURE	7
6.CHALLENGES	9
7.DRAWBACKS	10
8.CONCLUSION	10
9.FUTURE SCOPE	11
10.REFERENCES	12

PROBLEM STATEMENT

Modern web applications and network pipelines handle massive file transfers continuously. Inefficient data transmission increases bandwidth consumption, slows down network request-response cycles, and places a heavy burden on storage infrastructure. To address this, developers must leverage compression techniques.

The problem addressed in this project is to build an integrated platform that seamlessly handles file compression/decompression while systematically analyzing algorithm performance. The system must process files across a diverse size spectrum, evaluate the efficiency of custom frequency-based vs. native dictionary-based algorithms, log metadata metrics (such as execution time and percentage reduction), and graph the performance data on a logarithmic scale. This demonstrates the practical application of Design and Analysis of Algorithms (ADA) in solving real-world network and storage bottlenecks.

OBJECTIVES

- ☐ To develop a full-stack system that efficiently compresses and decompresses uploaded files.
- ☐ To apply the concepts of Design and Analysis of Algorithms (ADA) in solving a real-world data optimization and storage problem.
- ☐ To implement a custom Huffman Coding algorithm along with native Gzip and Brotli integrations for comparative analysis.
- ☐ To reduce network payload sizes and overall server-side memory overhead during file transfers.
- ☐ To analyze and benchmark algorithm performance based on execution speed (ms) and data density reduction.
- ☐ To generate interactive mathematical data visualizations mapping original versus compressed file sizes using Matplotlib.
- ☐ To demonstrate the practical importance of advanced text-encoding and dictionary-based heuristics in web engineering.

PROPOSED SOLUTION

The proposed system, is designed to automate and analyze data compression pipelines via an interactive, full-stack web application. The platform accepts user-provided files through an intuitive client UI, passes the binary buffers to a backend processing tier, and runs the user's algorithm of choice. Upon completion, the system returns the processed file along with detailed performance telemetry.

By migrating operations to a streamlined architecture, the platform removes manual data conversion tasks, preserves perfect data integrity, and delivers immediate evaluation results.

ALGORITHM USED:

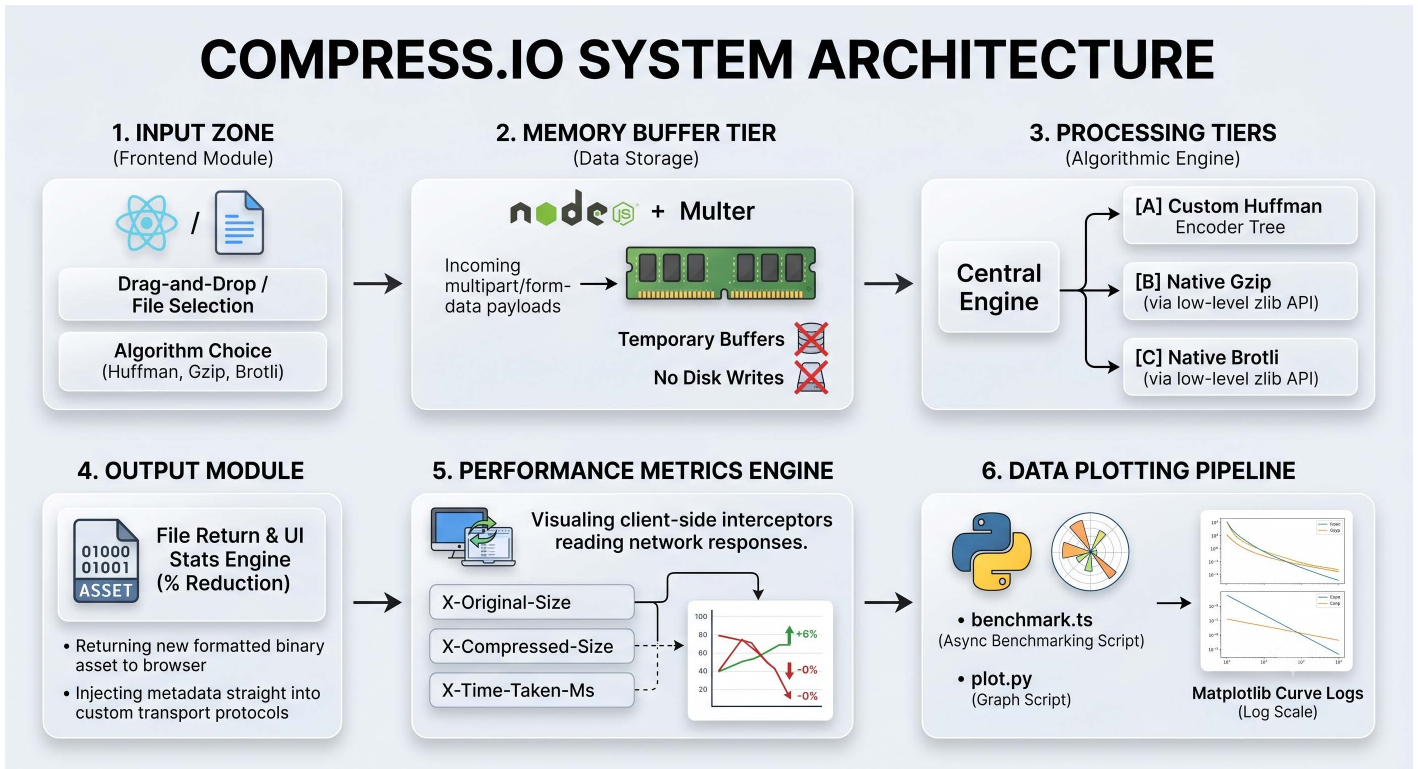
The system evaluates three core approaches representing different evolutionary steps in algorithmic design:

1. Huffman Coding (Custom Implementation)
 - Constructs a binary tree where characters with high frequencies reside closer to the root, assigning them shorter prefix-free codes.
 - Steps:
 - Parse the input buffer to calculate byte frequencies.
 - Build a priority queue/min-heap of node structures.
 - Recursively combine nodes to build the Huffman Tree.
 - Traverse the tree to generate unique binary sequences.
 - Encode the original file content with the shorter codes.
2. Gzip / DEFLATE (Node.js Native zlib)
 - A combination of the dictionary-based LZ77 sliding window technique and Huffman encoding.
 - Replaces repeated sequences of strings with length-distance references.
3. Brotli (Node.js Native zlib)
 - An advanced contextual compression algorithm using a static, pre-defined dictionary alongside an expanded sliding window size.

Advantages of Proposed System

- ☐ Highly effective across varying data types and file scales.
- ☐ Faster processing utilizing optimized in-memory streaming pipelines.
- ☐ Leverages diverse algorithmic strategies (Entropy-based vs. Sliding-Window vs. Context-Modeling).
- ☐ Perfectly tailored for modern web application performance tuning and asset distribution.

COMPRESS.IO SYSTEM ARCHITECTURE



Description of Architecture

The architecture of Compression consists of six major modules that work together to upload, temporarily buffer, compress, and analyze files using diverse algorithmic paradigms:

1. Frontend Module

A React-based drag-and-drop interface styled with glassmorphism that gathers the target file and the user's preferred compression technique.

2. Memory Buffer Tier

Handled by Node.js and Multer. It holds incoming multipart/form-data payloads completely in RAM as temporary buffers, avoiding performance-degrading disk writes.

3. Processing Tiers (Algorithmic Engine)

The heart of the platform. Executes either the custom Huffman encoder tree structure or invokes the low-level zlib API configurations for Gzip and Brotli.

4. Output Module

Returns the newly formatted binary asset back to the web browser while injecting metadata values straight into custom transport protocols.

5. Performance Analysis Module

Extracted by client-side interceptors reading X-Original-Size, X-Compressed-Size, and X-Time-Taken-
Ms network responses to showcase percentage changes in real-time.

6. Data Plotting Pipeline

An asynchronous benchmarking script (benchmark.ts) and graph script (plot.py) that pass structured testing metrics into Python's Matplotlib library to produce comparative charts.

Challenges

1. Algorithmic Parity Tracking

Ensuring clear benchmark evaluations across completely different architectural execution paths (custom JavaScript tree loops vs. pre-compiled native zlib binaries).

2. Buffer Control Constraints

Safely loading raw file inputs directly into system RAM using Multer without allowing memory leaks or causing server instances to crash.

3. State Association Maintenance

Keeping precise byte configuration headers aligned with compressed streams so unpacking files reproduces exact, uncorrupted originals.

4. Scale Plotting Discrepancies

Properly mapping processing variances when input files stretch from tiny sizes (100 Bytes) to large files (5 Megabytes), requiring log-scale adjustment layouts.

5. Environment Timing Standardization

Getting accurate execution logs (X-Time-Taken-Ms) that stay consistent despite background CPU scheduling tasks.

6. Strict Type Handling

Managing incoming binary arrays across full-stack boundaries safely using TypeScript..

DRAWBACK

1. The custom-built Huffman encoder struggles to identify multi-character repetitions, dropping efficiency compared to sliding-window models.
2. The engine relies entirely on in-memory allocation buffers, preventing it from processing large file formats above 100MB without causing out-of-memory errors..
3. Initial processing overhead for contextual algorithms like Brotli can occasionally introduce slight system latency on smaller text files.
4. The system is built to process single isolated items at a time and lacks parallel multi-file batch execution queues.
5. Custom data tracking remains stateless and resets with every application refresh because it does not have a database layer.

CONCLUSION:

The Compression platform successfully demonstrates the practical application of Design and Analysis of Algorithms (ADA) in network distribution and database space management. The project automates file manipulation tasks using a clean web design, making complex computer science paradigms transparent and easy to use.

By testing frequency-based code trees against sophisticated look-ahead dictionaries, the platform clearly highlights the performance characteristics of different algorithm types.

The underlying Matplotlib scripts visually reinforce core asymptotic concepts by mapping performance behavior over exponential data scaling. Ultimately, the system fulfills its goal of serving as a responsive, reliable benchmarking platform, validating the practical worth of sound algorithmic selection within professional software engineering pipelines.

FUTURE SCOPE:

In the future, the system can be enhanced with additional features to improve its functionality and scalability.

1. Database Integration

Adding structured storage layers to log historical execution data and support long-term trend analysis.

2. Stream Processing Architecture

Migrating the backend engine from full memory-buffer loading to continuous file streams to support multi-gigabyte data sets.

3. Parallel Execution Pools

Implementing background web worker threads and CPU clustering techniques to speed up concurrent file transformations.

4. From-Scratch LZ77 Implementation

Building a native sliding-window reference encoder by hand to remove vendor dependencies and broaden academic comparison depth.

5. Expanded Asset Analytics

Developing interactive performance dashboards to break down internal text pattern weights and file entropy profiles.

REFERENCES:

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C., *Introduction to Algorithms*, 4th Edition, MIT Press, 2022.
- Huffman, D. A., "A Method for the Construction of Minimum-Redundancy Codes", Proceedings of the IRE, 1952.
- Ziv, J. and Lempel, A., "A Universal Algorithm for Sequential Data Compression", IEEE Transactions on Information Theory, 1977.
- Node.js Standard Library documentation for the Native Zlib Compression API.
- Course materials and lecture notes for Design and Analysis of Algorithms (ADA)